

Congestion Minimization using Fog-deployed DRL-Agent Feedback enabled Traffic Light Cooperative Framework

1st Anuj Sachan

Department of Comp. Sci. and Engg.
Indian Institute of Technology Roorkee
Haridwar, Uttarakhand INDIA-247667
anuj_s@cs.iitr.ac.in

2nd Nisha Singh Chauhan

Department of Comp. Sci. and Engg.
Indian Institute of Technology Roorkee
Haridwar, Uttarakhand INDIA-247667
nisha_sc@cs.iitr.ac.in

3rd Neetesh Kumar

Department of Comp. Sci. and Engg.
Indian Institute of Technology Roorkee
Haridwar, Uttarakhand INDIA-247667
neetesh@cs.iitr.ac.in

Abstract—Congestion at signalized intersections can be alleviated by improving traffic signal control system's performance. In this context, Deep Reinforcement Learning (DRL) methods are increasingly gaining attention towards collaborative traffic signal control in vehicular networks for improving the traffic-flow. However, the existing collaborative methods lack in accounting the influence of neighbouring intersections traffic while working at a particular junction as built on the top of traditional client-server architecture. To address this, a Fog integrated DRL-based Smart Traffic Light Controller (STLC) cooperative framework is proposed via TCP/IP based communication among Fog node, Road Side Cameras (RSCs) and STLCs at the edge. The significant contributions of this work are: (1) A Fog node integrated DRL agent is proposed to minimize average waiting time and queue length, at the intersection, by generating Cycle Phase Duration (CPD) for the STLC via an appropriate coordination among neighboring intersections; (2) Utilizing the Fog node generated CPD as the feedback, a max-pressure based algorithm is proposed, for the STLC at the edge to improve the congestion at the intersection; (3) The performance of the proposed framework is analyzed on Indian cities OpenStreetMap utilizing the Simulation of Urban MObility (SUMO) simulator by varying arrival rate of the vehicles. The results demonstrate the effectiveness of the method over same line state-of-the-art methods.

Index Terms—Congestion, deep reinforcement learning, Fog computing, Intelligent Transportation System (ITS), smart city, traffic light scheduling, traffic light controller

I. INTRODUCTION

The increasing number of vehicles, despite limited physical transport infrastructure, endorse the need of Cyber-Physical System (CPS) framework to improve traffic flow in cities [1]. High vehicular density and inconsistent traffic conditions result in various environmental issues such as noise and air pollution, health issues i.e., tension and anxiety, etc. [2]. The traffic light controller (TLC) is responsible for maintaining the traffic flow at the intersections and optimizing various performance metrics such as waiting time, queue length, and throughput. However, an inefficient TLC can also lead to increased delays in various transportation services, such as

school buses and ambulances with an excessive fuel wastage [3]. IoT-enabled Intelligent Transportation System (IoT-ITS) deployment has significantly improved society's beliefs and the individual traveler experiences based on the promises of the advanced technologies involved in it. The primary objectives of IoT-ITS are to analyze, develop, integrate, and evaluate multi-purpose sensors, communication technologies, and algorithms to enhance traffic flow in the city [4]. As an outcome, it minimizes waiting time for various transport services including emergency services and improves environmental health, safety, and comfort for drivers, passengers, and pedestrians. With the advancements in IoT, Information & Communication Technology (ICT) and CPS have led to major improvements in traffic management systems, including: (i) A wireless communication-based system that makes it easier to install the TLC without need for communication cables across the city; (ii) The TLC can be operated not only at specific crossings, but also at various intersection types, ranging from simple single crossings with one light to complex intersections with multiple lights; (iii) Traffic congestion can be reduced by capturing real-time traffic states through IoT-enabled CPS and intelligent traffic light scheduling decision making [5].

Several traffic light control policies have been proposed by the researchers considering different scenarios [6] [7] [8]. The current TLC method based on maximum pressure [9] [10] has been proven to be effective and widely accepted. A max-pressure based TLC improves traffic flow by allocating phase duration to the road lanes with respect to the traffic load based on factors such as queue length and number of vehicles and reduces the congestion at the intersection. First adaptation of distributed max-pressure based TLC for urban transport networks is suggested by Varaiya [11] which is light-weight and can be run on IoT devices such as Raspberry Pi. Additionally, it meets scalability requirements as it is independent of the transport network size and topology, and it also offers stability by providing a bounded output when a bounded input is applied. In recent, reinforcement learning (RL)-based TLCs have been proposed [12]. Existing RL methods have attempted to improve the traffic flow at intersections by considering

This work is supported by SERB (Grant no: SER-EEQ/2019/000182) and CSIR (Grant no: 22-0817-19-EMR-II), Govt. of India.

real-time traffic information [13] [14], but they struggle to address high-dimensional traffic states. To overcome this issue, researchers have investigated deep reinforcement learning (DRL) algorithms [15] [16]. Further, the RL-based TLC that utilizes the max-pressure algorithm has been proven effective and widely accepted TLC [17]. However, the limitations of existing RL and DRL-based TLCs includes: (i) DRL methods in large-scale traffic networks become more complex when joint traffic information and Q-functions are introduced, (ii) In practice, existing distributed TLCs are limited to a single/local intersection, and they struggle in addressing collaborative traffic control decision making from multiple neighboring intersection. (iii) Most of the existing TLCs make routing decisions based on local traffic data in form of queue length (simple vehicle count) and do not account for the heterogeneity to make effective traffic control decisions.

To address the above-mentioned limitations, this work proposes a novel, Fog node integrated DRL-based traffic light control framework for smart transportation, and the significant contributions are listed as follows:

- The proposed Fog node integrated DRL-based agent takes the average waiting time (*AWT*) and average queue length (*AQL*) as the inputs from the corresponding intersection and generates the new cycle phase duration (*CPD*) by considering the traffic at the neighboring intersections.
- The proposed DRL agent executes on the Fog node, estimates the new *CPD* in advance, and sends the signal to the respective STLC. To overcome the communication latency between the Fog node and the controller, we introduce a latency parameter that can be adjusted as per the signal strengths and computational capacity of the Fog node.
- This framework utilizes the proposed max-pressure actuated algorithm at the edge computing node (Raspberry Pi) as the controller to generate effective traffic light control signals on a low-power computing node in real-time. To access the real-time traffic data, a well-known open-source simulator (SUMO) is utilized and run on a separate computing node at the edge.
- To measure the performance of the proposed Fog node integrated DRL (F-DRL) along with max-pressure algorithm at the edge (STLC), a prototyping simulation test-bed is developed on Indian city OpenStreetMap using Raspberry Pi, LED-based traffic-lights, and SUMO simulator [18]. The obtained results are compared with several same-line state-of-the-art algorithms, i.e., Multi-Mode TLC (MMTLC) [19], max-pressure based TLC (MPTLC) [3], and Fix Time Algorithm based TLC (FTATLC) [20], which proved the effectiveness of F-DRL STLC.

II. LITERATURE SURVEY

The Intelligent Transportation System (ITS) has emerged as the principal focus of smart city research [6]. Kamasetty et al. [21] designed a portable smart traffic signaling system

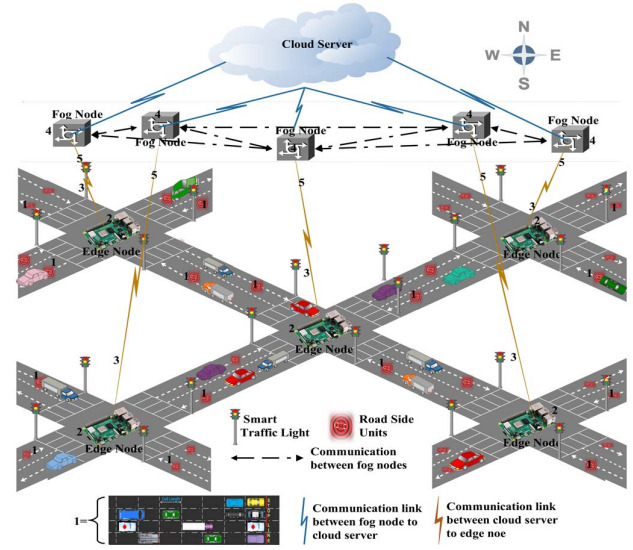


Figure 1: Congestion Optimization Network Framework

with cloud-assisted artificial intelligence utilizing a micro-controller (ESP32 dual core). There are four essential components in a typical traffic light control system: a brain-to-system controller to make control decisions, sensors to sense vehicles, Light Emitting Diode (LED) to display the traffic light phase (Red/Yellow/Green), and the countdown timer to count the remaining control signal phase duration [22]. V. Viswanathan et al. [23] proposed the wireless network and made an intelligent traffic light control system. The regulation of phase switching has been proposed using deep learning techniques. Varaiya [11] developed a TLC based on the max-pressure that requires estimation of queue length for the road lane. To do this, the authors developed a Discrete Event (DE) simulation to evaluate the performance of max-pressure as well as other standard controllers. Nilsson and Como [24] developed lipschitz continuous feedback-based system to control the traffic light signals by accounting for high and low traffic situations. Further, Pedro Mercader et al. [9] proposed a max-pressure based TLC considering the travel time for phase duration estimation. Most of the existing max-pressure based TLCs make traffic control decisions based on local traffic data in the form of travel time or queue length. Kumar et al. [16] developed CPS-based TLC using the DRL model. In this model, the environment is divided into two planes: the data plane, and the control plane. The data plane collects vehicular traffic information such as vehicle speed, position, etc., and sends it to the DRL-based agent. At the control plane, the central server generates a congestion prevention signal in the form of a traffic light control signal to manage traffic flow. Sachan et al. [10] developed a quantized variant of max-pressure utilizing software-defined networking. A fuzzy inference system is used to generate phase duration at the cen-

tralized controller. Recently, Borges et al. [25] also proposed a TLC based on Hierarchical Reinforcement Learning (HRL) and demonstrated the model's effectiveness using the SUMO simulation tool. In addition, Boukerche et al. [17] developed a traffic cooperative TLC based on Reinforcement Learning (RL) and max-pressure control algorithm.

From the above state-of-the-art methods it is observed that most of the controllers used queue length to estimate the pressure. To execute the Machine Learning (ML)-based models, high-end computing infrastructure is required. The type of high-end computing infrastructure used to execute ML-based models is a powerful central processing unit (CPU) or graphics processing unit (GPU) with a higher memory configuration. Besides, it is also observed that state-of-the-art methods remained unexplored to deploy on edge computing nodes to validate their computational feasibility in real-time. This work aims to minimize congestion at the intersection by proposing a cooperative traffic signal control framework at multiple intersections for improving the traffic flow. In this context, we designed a Fog-integrated DRL-based STLC to generate an effective cycle phase duration. The motivation for using Fog nodes instead of traditional high-end computing infrastructure is to provide a more cost-effective and scalable solution for executing ML models, especially in edge computing scenarios where vehicular data is generated and processed at the edge of the network. Fog nodes are closer to the data source, reducing the amount of data that needs to be transmitted to a central location for processing, which can reduce latency, improve response times, and lower the risk of data loss. Additionally, Fog nodes have relatively low power consumption and can be deployed in remote or resource-constrained environments, making them well-suited for edge computing applications (Fig. 1). In this framework, the phase duration to the respective road lane is assigned using an efficient maximum pressure algorithm, at the edge, to improve traffic flow at the intersection.

III. PROBLEM STATEMENT AND SOLUTION APPROACH

This work identifies the problem of cooperative traffic light scheduling problem by considering the imbalance distribution of vehicular traffic at the intersections. To do this, the relevant vehicular traffic parameters, i.e., vehicle type, count, speed, etc., are collected through the deployed RSCs [15] in the environment, as shown in Fig. 1. The identified problem becomes more complex by considering the dense network of intersections, frequent unexpected congestion, and uncertain vehicular dynamics. The specified problem is solved by proposing a Fog-integrated DRL-based Smart Traffic Light Controller (STLC) cooperative framework. Fog node integrated DRL agent is proposed considering average waiting time and queue length at the intersection to generate Cycle Phase Duration (CPD) for the STLC via appropriate coordination among neighboring intersections. Following this, using the Fog node generated CPD as the feedback for the STLC at the edge, a max-pressure based algorithm is proposed, to minimize the congestion at the intersection.

Commonly, the TLC produces three control signals, i.e., R/Y/G. A traffic light is not able to control traffic flow coming from multiple directions. Thus, numerous traffic lights must be deployed at the intersection and should work in synchronization. At the intersections, the traffic signals are required to be kept changed in such a way that it can maintain the non-conflicting flow from multiple directions. The phase duration denotes the time interval for staying at a particular signal (R/G/Y). At each intersection, all the phases are required to be changed cyclically, a cycle that keeps repeating. For the red signal, no vehicle is allowed to pass through it. On the contrary, vehicles in the respective lane have free passage for the green signal. The yellow signal means vehicles should slow down when the signal changes from green to red. The aim of this work is to optimize the congestion by generating the traffic light control signals considering the relatively highly congested regions in the urban city and balance the traffic flow w.r.t. several QoS parameters, i.e., queue length, waiting time, throughput, and speed at each intersection.

IV. THE PROPOSED FRAMEWORK

A. System Model

This sub-section models the set of traffic light intersections and the set of road lanes in the city using graph theory. Hence, the urban network is represented with the help of a directed graph $G=(V,E)$, where, V indicates the nodes (set of intersections of road lanes), and E indicates the links (set of road lanes). Each link is represented as an ordered pair $(r,s) \in E$. The set of input nodes over the link (r,s) is represented by $en_{r,s}$, and the set of output nodes is represented by ex_{s} . Here, the nodes are classified into three categories: (i) source nodes, (ii) intermediate nodes, and (iii) destination nodes. (i) Source nodes are those nodes from where vehicle enters into the network, (ii) intermediate nodes are the signalized intersection between source and destination nodes, and (iii) destination nodes allows vehicles to exit from the network. This study proposes a STLC that generates green, yellow, and red traffic light signals at the intermediate nodes in an optimised manner while reducing congestion. The model is event triggered, where the number of vehicles can be updated on the link (r,s) as:

$$UP_{r,s}(t+1) = UP_{r,s}(t) + (Y_{r,s}^{arrive}(t) - Y_{r,s}^{depart}(t)) \times CPD \quad (1)$$

where, t represents discrete time index for the traffic network model and its controller, CPD denotes the signal's cycle phase duration at the intersection, and initially, that is taken as the constant for each intersections of road lanes. Later, it is updated through DRL agent considering Average Queue Length (AQL) and Average Waiting Time (AWT). With this, the average traffic flow which arrives and departs the link (r,s) is modelled as:

$$Y_{r,s}^{depart}(t) = \sum_{x \in ex_{r,s}} Y_{r,s,x}^{depart}(t), \quad Y_{r,s}^{arrive}(t) = \sum_{z \in en_{r,s}} Y_{z,r,s}^{depart}(t) \quad (2)$$

Where,

$$Y_{r,s,x}^{depart}(t) = \min(O_{r,s,x}(t), U_{r,s,x}(t), X_{r,s,x}(t)) \quad (3)$$

$$O_{r,s,x}(t) = \frac{d_{r,s,x}(t) \times (p_{s,x} - u_{s,x}(t))}{PD \times \sum_{z \in \epsilon n_{s,x}} d_{z,s,x}(t)} \quad (4)$$

$$U_{r,s,x}(t) = QL_{r,s,x}(t)/CPD + Y_{r,s,x}^{arrival}(t) \quad (5)$$

$$X_{r,s,x}(t) = d_{r,s,x}(t) \times b_{r,s} \times q_{r,s,x}(t)/CPD \quad (6)$$

where, $O_{r,s,x}(t)$ denotes the over-saturated flow rate of traffic conditions, $U_{r,s,x}(t)$ denotes unsaturated flow rate of traffic conditions, and $X_{r,s,x}(t)$ denotes saturated flow rate of traffic conditions. Additionally, $d_{r,s,x}(t)$ represents the lump of traffic which turns from (r,s) to (s,x) via an outward input, $q_{r,s,x}(t)$ represents the green signal time of the traffic stream going from (r,s) towards x at timestamp t , $b_{r,s}$ represents the saturated flow rate which leaves (r,s) , and the capacity of the link (s,x) is denoted as $p_{s,x}$. The number of vehicles in the queue, i.e., Queue Length ($QL_{r,s,x}$) is updated as:

$$QL_{r,s,x}(t+1) = QL_{r,s,x}(t) + (Y_{r,s,x}^{arrival}(t) - Y_{r,s,x}^{depart}(t)) \times CPD \quad (7)$$

where, $Y_{r,s,x}^{arrival}(t)$ represents the flow rate towards x arriving at the end of queue, and it is given by:

$$Y_{r,s,x}^{arrival}(t) = d_{r,s,x}(t) \times Y_{r,s}^{arrival}(t) \quad (8)$$

$$Y_{r,s}^{arrival}(t) = (1 - v_{r,s}(t)/CPD) \times Y_{r,s}^{arrival}(t - \omega_{r,s}(t)) + v_{r,s}(t)/CPD \times Y_{r,s}^{arrival}(t - \omega_{r,s}(t) - 1) \quad (9)$$

$$TT_{r,s}(t) = (p_{r,s} - QL_{r,s}(t)) \times WF_c(t) / (k_{r,s}^{lane} \times SV_{r,s}^{free}) \quad (10)$$

$$\omega_{r,s}(t) = \lceil TT_{r,s}(t)/CPD \rceil, v_{r,s}(t) = TT_{r,s}(t) - \omega_{r,s}(t) \times CPD \quad (11)$$

where, $Y_{r,s}^{arrival}(t)$ is the vehicles' flow rate arriving at the end of the queue on link (r,s) ; $TT_{r,s}(t)$ is the travel time required for a vehicle to reach the end of the waiting queue on link (r,s) at time step t ; WF_c is the weight factor of each vehicle type. The $k_{r,s}^{lane}$ is the road lanes' number on link (r,s) , and $SV_{r,s}^{free}$ is the free flow speed of the vehicles on link (r,s) . Following this, the Average Queue Length (AQL) at any intersection is calculated as:

$$AQL_{intersection}(t+1) = \sum_{k \in n} \left(\frac{QL_{r_k,s_k,x_k}(t)}{C_k} \right) \quad (12)$$

where, n represents the number of roads intersecting at the intersection, and C represents the total number of vehicles. The waiting time (WT) of the vehicles towards x is updated:

$$WT_{r,s,x}(t) = WW \times CPD \quad (13)$$

$$WW = ((t - TT_{r,s}(t)/PD) \times Y_{r,s}^{arrival}(t) - Y_{r,s,x}^{depart}(t))$$

Following this, the Average Waiting Time (AWT) at any intersection is calculated as:

$$AWT_{intersection}(t+1) = \sum_{k \in n} \left(\frac{WT_{r_k,s_k,x_k}(t)}{C_k} \right) \quad (14)$$

Based on this system model, the obtained AQL and AWT are computed. These parameters are utilized as a state to the Fog node integrated DRL agent. Following this, the agent generates the action as Cycle Phase Duration (CPD). The generated CPD is used as feedback to the STLC at the edge.

B. Communication Model

As the number of vehicles continues to grow, vehicular wireless network technology is increasing simultaneously. Dedicated Short Range Communication (DSRC) can be used when Road Side Cameras (RSCs) capture the vehicles' information and forward to the fixed Edge-Node (ENO). The IEEE802.11p is used for DSRC [26]. Further, to improve the quality of service delivered to fixed ENO to Fog-node, LTE and 5G can be utilized.

1) *Communication model of ENO to Fog-node*: According to the present study, the transmission link between the ENO and the Fog-node is set up as a flat Rayleigh fading channel, irrespective of channel inference, and the wireless channel bandwidth is also equally distributed among the fixed ENO and Fog-node. From task generation to driving out of reach of the ENO to Fog-node, the average transmission rate (atr) of the upload link can be calculated using the Shannon formula:

$$\overline{atr_{fn}^{ENOtoFog-node}} = \frac{BW^{ENOtoFog-node}}{m} \times Q \quad (15)$$

$$Q = \log_2 \left(1 + \frac{UPow_{fn}^{n,up} \cdot CG^{ENOtoFog-node}}{\delta^2} \right)$$

where, $BW^{ENOtoFog-node}$ represents the ENO's bandwidth to Fog-node's upload link, $UPow_{fn}^{n,up}$ denotes fn_{th} ENO to the n_{th} Fog-node upload power, $CG^{ENOtoFog-node}$ represents the channel gain linking between fixed ENO and Fog-node, and δ^2 represents noise power in the background.

2) *Communication model of RSC to ENO*: In this work, RSC to ENO communications are carried out via orthogonal frequencies, which reduces the RSC interaction. RSC to ENO data transfer rates can be demonstrated as follows:

$$\overline{atr_{rc}^{RSCtoENO}} = BW^{RSCtoENO} \times K \quad (16)$$

$$K = \log_2 \left(1 + \frac{UPow_{rc}^{k,up} \cdot CG^{RSCtoENO}}{\delta^2} \right)$$

where, $BW^{RSCtoENO}$ represents bandwidth between the RSCs and fixed ENO, $UPow_{rc}^{k,up}$ denotes rc_{th} RSC to the k_{th} ENO upload power, $CG^{RSCtoENO}$ represents the channel gain linking between road side camera and fixed ENO. Considering the above communication model (Eqs. 15 and 16), all flows between ENO to Fog-node and RSCs to ENO are formulated.

the junction are optimized. As a result, the reward obtained by the agent is scored twice: once before and once after the action is taken. The following equations are used to calculate the reward for the agent:

$$Re_{final} = Re_b - Re_a \quad (20)$$

Here, Re_b represents the reward calculated before the action is performed whereas Re_a is the reward evaluated after the action is performed. Further, the values of Re_b and Re_a can be evaluated using equations 21 and 22 respectively.

$$Re_b = \sum_{k=1}^n WF_k * (NWT_k + NQL_k) \quad (21)$$

$$Re_a = \sum_{k=1}^n WF_k * (NWT'_k + NQL'_k) \quad (22)$$

$$Zv = \frac{Zv_{curr} - Zv_{min}}{Zv_{max} - Zv_{min}} \quad (23)$$

where, n denotes the total number of lanes at the junction, WF_k is the weight of the vehicles on k_{th} lane. The NWT_k and NQL_k are the normalized waiting time and queue length of the k_{th} lane respectively. The NWT'_k and NQL'_k are the new normalized waiting time and queue length of the k_{th} lane respectively that is evaluated after the action is performed. Positive rewards indicate that the chosen action is better carried out for the given the current situation of the traffic at the intersection. The Zv is the normalized variable for the respective parameter, Zv_{curr} , Zv_{min} and Zv_{max} denotes the current, minimum, and maximum value of the normalized traffic parameter Zv .

E. Deep Neural Network

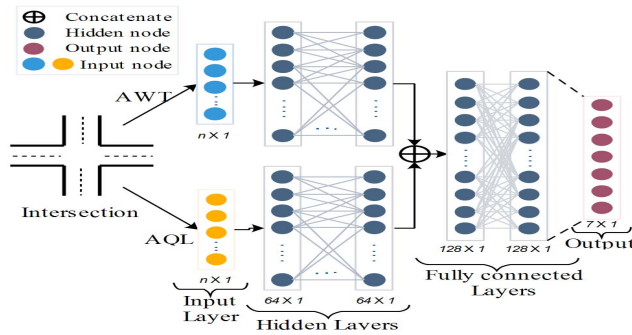


Figure 3: The Proposed Deep Neural Network Architecture; AWT: Average Waiting Time; AQL: Average Queue Length; n: number of lanes at an intersection

Fig. 3 shows the layered architecture of a deep neural network (DNN). The proposed model takes the AWT and AQL as input from the junction. Both the input vectors are of size $n \times 1$, where n represents the total number of lanes at a particular junction. After the input construction, both inputs are sent to the respective neural network. Each neural network is constructed using multiple hidden layers stacked together.

The size of each hidden layer is 64×1 , and the activation function used is a leaky Rectified Linear Unit (ReLU). The ReLU function is represented using the following equation:

$$f(x) = \begin{cases} x & x \geq 0 \\ \beta x & x < 0 \end{cases} \quad (24)$$

Further, the outputs from the hidden layers are concatenated to form a new vector of extracted features of size 128×1 . The output obtained after concatenation is fed to a fully connected layer with ReLU as an activation function. The last layer of the fully connected layer uses softmax as the activation function. The equation for the softmax function is given by Eq. 25, where, x represents the input vector, x_k is the k_{th} element of the input vector, and c is the number of possible outcomes (number of classes).

$$Sm(x_k) = \frac{\exp(x_p)}{\sum_{p=1}^c \exp(x_p)} \quad (25)$$

The fully connected layers output the Q-value corresponding to both possible actions, an agent can perform. The parameters of the deep neural network are initialized with random weights. At the beginning of every episode, the simulated traffic environment is analyzed by the agent, and the AWT and AQL vectors are drawn out to create the state space St_t of a particular junction. Using the inputs, the agent performs an action Ac_t with the highest Q-value. After the action Ac_t , agent is awarded some reward Re_t according to the likeliness of the performance. A new state St_{t+1} is received after the chosen action is performed. A 4-tuple $\langle St_t, Ac_t, Re_t, St_{t+1} \rangle$ a training example is stored in the experience replay memory, and it is used to train the deep neural network. There is a memory threshold limit that leads to the replacement of old memories by new ones. This way, the deep neural network is trained on the experiences St_t, Ac_t . The model aims to learn the mapping between action space and corresponding state space by minimizing the loss function ($J(\theta)$). Here, the mean squared error is used as the loss function which is given by the equation below:

$$J(\theta) = \frac{1}{D} \sum_{t=1}^D [(Re_t + \alpha max_{a'} Q(s_{t+1}, a', \theta) - Q(s_t, a_t, \theta))^2] \quad (26)$$

where, D is the training dataset size. The weight parameters (θ) are updated using stochastic gradient descent called Root Mean Squared Propagation (RMSProp).

F. Algorithm

Algorithm 1: The algorithm starts by taking specific inputs, as mentioned in Algorithm 1. Initially, Algorithm 1 is deployed at each g_{th} traffic light junction. Based on the inputs, the DRL agent is trained by calling $TrainDRLAgent(AWT_g, AQL_g)$ and deployed at the respective Fog node that is explained in detail by Algorithm 2. The $Step_g$ and total cyclic phase duration ($CPD_{initial}$) are initialized to 0 and 60 seconds, respectively, where $1 \leq g \leq G$. Before going to further simulation steps, $CPD_{g_{old}}$ is updated by subtracting μ seconds to handle

the latency caused during the communication and processing latency. The value of μ can be adjusted according to signal strengths and computational capacity of the Fog node. First,

Algorithm 1 Main-Algorithm

Input: Total Number of Intersections (G), Total Number of Vehicles (C), Vehicular Network of the city.

Output: Traffic light control phase (Green/Yellow/Red).

Notation:

$CPD_{initial}$: Total cycle phase duration of an intersection

$WF[]$: Weight Matrix for g_{th} intersection

μ : Communication-Processing latency

Initialize: $CPD_{initial} \leftarrow 60$, $Step_g \leftarrow 0$, $k \leftarrow 0$, $t_g \leftarrow 0$, $i \leftarrow 0$, $\mu \leftarrow 5$;

```

1: for  $g \in G$  do
2:   TrainDRLAgent( $AWT_g$ ,  $AQL_g$ );
3:   Deploy the trained agent at  $g_{th}$  junction
4:    $CPD_{gold} \leftarrow (CPD_{g_{initial}} - \mu)$ 
5:   while  $Step_g < SimulationSteps$  do
6:     if  $(t_g \% (CPD_{gold})) == 0$  then
7:        $VID_g = getVehicleList(C_g)$ 
8:        $l_{type} = vehicle.getTypeID(VID_g)$ 
9:       for  $i \in C_g$  do
10:         $(WF_g[i]) = \begin{cases} WF_{car}, & \text{if } l_{type} = Car \\ WF_{bus}, & \text{if } l_{type} = Bus \\ WF_{mbk}, & \text{if } l_{type} = Motorbike \\ WF_{ambu}, & \text{if } l_{type} = Ambulance \\ WF_o, & \text{otherwise} \end{cases}$ 
11:       end for
12:       Compute  $AQL_{g,k}$  Using Eq(12);
13:       Compute  $AWT_{g,k}$  Using Eq(14);
14:       SendMsgToFN( $AQL_g$ ,  $AWT_g$ ,  $CPD_{gold}$ );
15:        $CPD_{g_{new}} \leftarrow DRL\_CPD\_Generator(AWT_g, AQL_g, CPD_{gold});$ 
16:       RecvMsgFromFN( $CPD_{g_{new}}$ );
17:       SendMsgToENO( $AQL_g$ ,  $AWT_g$ ,  $CPD_{g_{new}}$ );
18:       STLC( $AWT_g$ ,  $AQL_g$ ,  $CPD_{g_{new}}$ );
19:     end if
20:      $t_g + 1$ ;
21:      $Step_g + 1$ 
22:   end while

```

the vehicle list is extracted by using $getVehicleList(C_g)$ and stored into VID_g , and later the vehicle type is extracted by using $vehicle.getTypeID(VID_g)$. After that $WF_g[]$ is the set of weights for all the vehicles presented on all the lanes at g_{th} intersection. Based on $WF_g[]$, average queue length ($AQL_{g,k}$) and average waiting time ($AWT_{g,k}$) are estimated using Eq. (12) and Eq. (14) respectively for k_{th} lane of g_{th} intersection. Next, the computed parameters are sent to the Fog node (FN) using $SndMsgToFN(AWT_g, AQL_g, CPD_{gold})$. Based on these parameters, the $CPD_{g_{new}}$ is updated by utilizing $DRL_CPD_Generator(AWT_g, AQL_g)$, that is re-

ceived by STLC i.e., $RecvMsgFromFN(CPD_{g_{new}})$ at g_{th} intersection. The detailed pseudo-code for generating $CPD_{g_{new}}$ is presented in Algorithm 3. Following this, updated parameters are sent to edge node (ENO) using $SndMsgToENO(AWT_g, AQL_g, CPD_{gold})$. Based on these parameters, traffic light phase is updated by setting phase duration using $STLC(AWT_g, AQL_g, CPD_{g_{new}})$ that is detailed in Algorithm 4. After that t_g is incremented by one. The similar procedure is repeated for number of simulation steps.

Algorithm 2 TrainDRLAgent

Input: AWT_g , AQL_g , Number of Epochs (E), Learning Rate (α), Replay Memory Size (M), Discount Factor (γ), Exploration Rate (ϵ).

Notation:

θ : Deep neural network weight parameter

St : State

AWT_g : Average Waiting Time at junction g

AQL_g : Average Queue Length at junction g

m : Replay Memory

Re : Performed action reward

```

1: for each  $e \in E$  do
2:   Initialize the TraCI environment as  $Evrmt$ ;
3:    $St \leftarrow getPresentState(IID)$ ;
4:   for each  $steps \in SimulationSteps$  do
5:      $Ac \leftarrow agent.getAction(St)$ ;
6:      $Re \leftarrow getReward(WF_g[], St)$ ;
7:      $St' \leftarrow getNextState(IID)$ ;
8:     if  $|m| < M$  then
9:       Delete the oldest data from  $m$ ;
10:    end if
11:    Append  $\langle St, Ac, Re, St' \rangle$  in  $m$ ;
12:     $J = \frac{1}{M} \sum_{t=1}^M [(Re_t + \alpha \max_{Ac'} Q(St_{t+1}, Ac', \theta) - Q(s_t, a_t, \theta))^2]$ ;
13:    Utilizing the RMSProp algorithm, update  $\theta$  with  $J$ ;
14:    Update  $\epsilon$  value
15:  end for
16: end for

```

Algorithm 2: *TrainDRLAgent* function is used for training the DRL agent. It takes AWT_g , AQL_g , hyperparameters such as number of epochs (E), learning rate (α), replay memory size (M), discount factor (γ), and exploration rate (ϵ) as the inputs. SUMO-TraCI is used to initialize the simulation environment. The $getPresentState(IID)$ saves the present state of the intersection as St , which is a tuple consisting of AWT_g and AQL_g , where IID represents the intersection ID in the network. The deep neural network's parameters are updated at each simulation. The $agent.getAction(St)$ returns the agent's action based on the current traffic state. The $getReward(WF_g[], St)$ returns a reward for the action performed based on the corresponding vehicles' weight matrix ($WF_g[]$) and the present state (St). After an action is performed, the next state St' is generated by $getNextState(IID)$. The replay memory saves the four-tuple

$\langle St, Ac, Re, St' \rangle$. The cost function (J) is now computed and utilized to update DNN parameters using the RMSProp.

Algorithm 3 DRL_CPD_Generator

Input: $AWT_g, AQL_g, CPD_{g_{old}}$

Output: Updated $CPD_{g_{new}}$

Initialize: $change=[10, 20, 30, 10, 20, 30]$;

```

1: for each  $g \in G$  do in parallel
2:    $A_g \leftarrow agent.getAction(AWT_g, AQL_g)$ ;
3:   if  $A_g == 0$  or  $A_g == 1$  or  $A_g == 2$  then
4:      $CPD_{g_{new}} = CPD_{g_{old}} + (change[A_g] * CPD_{g_{old}})/100$ ;
5:   end if
6:   if  $A_g == 3$  or  $A_g == 4$  or  $A_g == 5$  then
7:      $CPD_{g_{new}} = CPD_{g_{old}} - (change[A_g] * CPD_{g_{old}})/100$ ;
8:   end if
9:   if  $A_g == 6$  then
10:     $CPD_{g_{new}} = CPD_{g_{old}}$ ;
11:  end if
12: end for
13: return  $CPD_{g_{new}}$ 

```

Algorithm 4 STLC

Input: $AWT_g, AQL_g, CPD_{g_{new}}$

Output: Traffic light control phase (Green/Yellow/Red).

Notation:

MRP : Maximum road pressure

Initialize:

$\chi \leftarrow 0.5$; $\triangleright$ Queue length factor
 $\psi \leftarrow 0.5$; $\triangleright$ Waiting time factor

```

1: for each  $k \in n_g$  do in parallel
2:    $MRP_{g,k} = \chi \times AQL_{g,k}(t_g) + \psi \times AWT_{g,k}(t_g)$ ;
3: end for
4: for  $k \in n_g$  do
5:    $SetGreenSignal(laneID_k, \frac{MRP_{g,k} \times CPD_{g_{new}}}{\sum_{i \in C_{g,k}} MRP_{g,i}})$ ;
6:    $SetRedSignal(laneID_{\xi}, \frac{MRP_{g,k} \times CPD_{g_{new}}}{\sum_{i \in C_{g,k}} MRP_{g,i}})$ ;
    $\triangleright$  where,  $1 \leq \xi \leq n_g$ , and  $l \neq k$ 
7:    $SetYellowSignal(laneID_{n_g}, 3)$ ;
8: end for

```

Algorithm 3: $DRL_CPD_Generator(AWT_g, AQL_g, CPD_{g_{old}})$ executes when it receives the traffic parameters i.e., AWT_g, AQL_g , and $CPD_{g_{old}}$ from the g_{th} junction. A temporary list named *change* is initialized which represents the percentage by which the phased duration is to be updated. The first half elements of the list are the percentage by which the CPD is increased whereas the second half is the percentage by which the CPD is decreased. For the g_{th} intersection, $CPD_{g_{new}}$ is calculated based on the action taken by the agent while receiving the values of AWT_g, AQL_g and $CPD_{g_{old}}$. **Algorithm 4:** The $STLC(AWT_g, AQL_g, CPD_{g_{new}})$, at the edge, is used to set/update the traffic light control signal phase (Green/Yellow/Red) duration. The AQL and AWT are set

with equal weightage using $\psi = 0.5$ and $\chi = 0.5$, respectively. Using this, maximum road pressure ($MRP_{g,k}$) is measured for each lane at the respective intersection. However, the values of ψ and χ can be altered as per application requirements, where, $\alpha + \beta == 1$. Here, k is the loop variable ($0 \leq k \leq n_g - 1$) to represent the number of road lanes at g_{th} intersection, where, n_g is the total number of road lanes at g_{th} intersection. Based on $MRP_{g,k}$ and $CPD_{g_{new}}$, signal phase (Green/Yellow/Red) duration is proportionally set/updated for each lane. After completing one phase (Green/Red) and before switching to the next phase, the yellow phase duration is set to 3 seconds which indicates phase switching is about to take place.



Figure 4: Map snapshot SUMO for simulation study

V. SIMULATION STUDY

A. Simulation-based Prototype Test-bed Setup

In order to generate real-time traffic, SUMO simulation is prepared on an Indian city's OpenStreetMap as depicted in Fig. 4. A test-bed setup employing a high-end server as Fog Server, IoT devices such as Raspberry Pis, LEDs, Jumper Wires, HDMI, and display screen, is developed as shown in Fig. 5. A well-known open-source simulator, i.e., Simulation of Urban MObility (SUMO) [18] is used to generate the real-time traffic. In SUMO, the loop detectors are used to detect the presence of vehicles on the road. The loop detectors extract information such as vehicle count, speed, and occupancy. This information is required for the simulation considering real-time traffic conditions. Fig. 5 shows the test bed developed to perform the experiments. The SUMO simulation is set on a 64-bit PC with Core i7-9700 with 3.00 GHz processor and 16 GB RAM. The SUMO sends the AWT and AQL to the Fog node that is created on a server (64-bit with Core Xenon(R) Silver 4215 with 2.5GHz processor and 32 GB RAM) using Oracle VirtualBox. Further, the Raspberry Pi 3 Model B+ is used [28] with a 1.4 GHz 64-bit quad-core processor with 4GB RAM, full-size High-Definition Multimedia Interface(HDMI) port, 40 GPIO pins [28], two 3.3V pins, two 5V pins, and eight ground pins. A power input of 5V/2.5A DC is required to operate. In particular, LED connections are set up as follows: for the 'East' direction, GPIO 9, 10, 11, and GND pin; for the 'West' direction, GPIO 2, 3, 4, and GND pin, for the 'North' direction, GPIO 16, 20, 21, and GND pin, and for the 'South' direction, GPIO 7, 8, 25, and GND pin are utilized. Further, to test the STLC, real-time traffic states are fed to STLC from SUMO-generated simulation. STLC is implemented in the python 3.8 version, and the SUMO traffic simulation is interfaced using TRACI to control the traffic flow and generate the traffic light control signals according

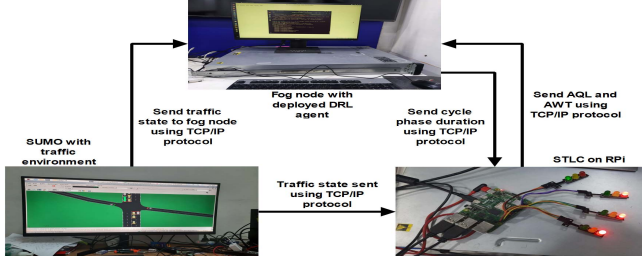


Figure 5: Test bed Setup

Table I: Simulation Parameters

SUMO Simulation Parameters	Value
Length of Motorbike	2.4m
Length of Car	3.4m
Length of Bus/Truck	8m
Length of other vehicles	6m
Max. Lane Speed	22m/s
Max. Speed of MotorBike	11m/s
Max. Speed of Car	13m/s
Max. Speed of Bus/Truck	9m/s
Max. Speed of other vehicles	14.3m/s
Turn-speed limit of all vehicles	6.3m/sec
Max. Acceleration of MotorBike	3.1m/sec ²
Max. Acceleration of Car	3.3m/s ²
Max. Acceleration of Bus/Truck	0.7m/sec ²
Max. Acceleration of other vehicles	2.3m/sec ²
Max. De-acceleration of all Vehicles	3.0m/sec ²
Driving Proficiency(λ)	0.4
Detector Frequency	60
Min. Gap between Two Vehicles	3.0m
Road lane type	Unidirectional
Binomial Distribution (INT)	2
Latitude of the map	26°13'5.8332" N
Longitude of the map	78°10'58.1916" E

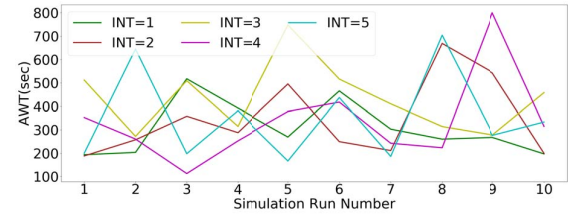
to STLC. The output (R/G/Y) of the STLC is displayed on LEDs connected to the Raspberry Pi GPIO pins. The STLC at the edge continuously collects real-time traffic states from the SUMO and computes the parameters: AWT and AQL. These parameters are communicated to the Fog server using TCP/IP protocol that is used as the state to the DRL agent (deployed on the Fog server), and the DRL agent generates the action to produce an appropriate CPD for STLC. Following this, STLC computes output (traffic light signals R/Y/G) to the LEDs.

Simulation Parameters: To implement the proposed F-DRL STLC, all the modules are programmed in Python, and vehicular network simulation is developed in SUMO. An OpenStreetMap of Gwalior City is downloaded as .osm file that is converted into a network of road lanes and traffic light intersections using the net-convert tool. SUMO offers a Traffic Control Interface (TRACI) as one of its modules to allow users to control the traffic behavior in the simulation via python implementation. The integration of SUMO and Python is quite successful with the aid of TRACI; this allows for the control of SUMO traffic flow via algorithmic code written in Python. Table I represents the SUMO traffic simulation parameters. For training the DRL the following parameters were used: Learning Rate (α) = 0.01, Replay Memory Size (M) = 10^6 , Discount Factor (γ) = 0.99, Exploration Rate (ϵ) =

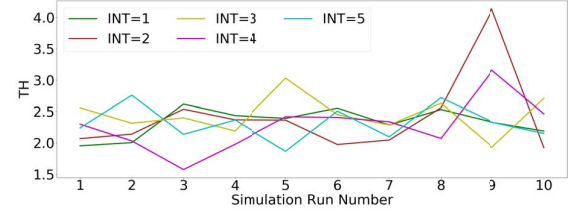
1.0, Number of hidden layers = 4, Number of fully connected layers = 2, Leaky ReLU parameter (β) = 0.01, and Number of Epochs (E) = 50.

B. Experimental Results

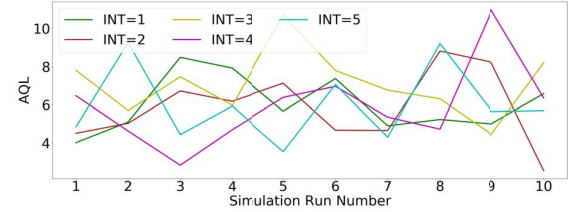
Several performance metrics, like, ThroughPut (TH), Average Speed (AS), Average Waiting Time (AWT), and Average Queue Length (AQL) are used to evaluate the effectiveness of the proposed F-DRL STLC. Despite being validated on a real-time test bed, the F-DRL STLC is also compared with other traffic light controllers of a similar kind, including the MMTLC [19], MPTLC [3], and FTATLC [20] to verify its effectiveness. The X-axis and Y-axis represent the number of episodes and the performance metric respectively. The "number of episodes", at the X-axis, represents the number of SUMO simulation iterations.



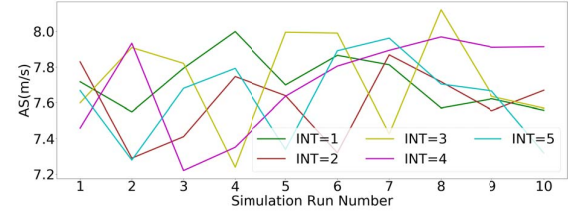
(a) Results on average waiting time



(b) Results on average throughput



(c) Results on average queue length



(d) Results on average speed

Figure 6: Results on varied maximum number of simultaneous arrivals of the vehicles

1) *Performance Under Simultaneous Vehicles' Arrivals:*
The randTrips.py script is used to generate traffic on the

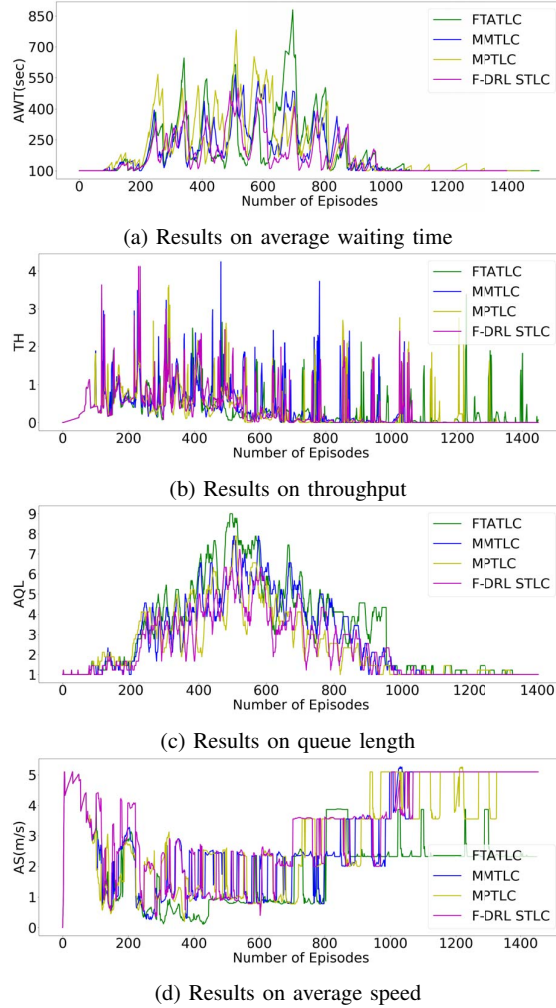


Figure 7: Comparative results of Proposed Algorithm

vehicular road network drawn from OpenStreetMap in the SUMO simulation. This script has two ways to generate traffic: i) an option `-period <FLOAT>` (default 1 vehicle per second) to control the arrival rate, and ii) the exact arrival rate can also be randomized using the Binomial Distribution (BD) with a parameter *INT*, representing the maximum number of simultaneous arrivals. To represent real traffic scenarios, this study uses the second option i.e., BD, to generate traffic on the road lanes. The performance of the F-DRL STLC is evaluated through a set of experiments with 10 repeated simulations and random seeds. Fig. 6(a), (b), (c), and (d) present the results for AWT, AQL, AS, and TH, respectively. The value of *INT* is varied from 1 to 5, and the results show that the F-DRL STLC performance remains acceptable even with increased arrival rates. The value of *INT* is set to 2 for further experimentation in SUMO, demonstrating the viability of the proposed F-DRL STLC in dense traffic.

2) *Comparative Study*: In this sub-section, the comparative study is carried out to analyze the comparative performance

behavior of the F-DRL STLC with other same-line state-of-the-art methods. The *AWT* results are shown in Fig. 7(a). Results show that the F-DRL STLC minimizes AWT by approximately 26.33%, 15.57%, and 37.65% in comparison to MMTLC [19], MPTLC [3], and FTATLC [20] respectively. For the same experimental settings, throughput (*TH*) is also evaluated in Fig. 7(b). The results in Fig. 7(b) show that the F-DRL STLC improves by 9.07%, 4.76%, and 13.39% over MMTLC, MPTLC, and FTATLC respectively. Further, Fig. 7(c) depicts the AQL value at different episodes. The proposed method also reduces the average queue lengths by 22.3%, 16.21%, and 38.09% over MMTLC, MPTLC, and FTATLC respectively. Fig. 7(d) presents the value of average speed at different episodes. F-DRL STLC also beats MMTLC, MPTLC, and FTATLC methods in terms of *AS* with 25.36%, 18.34%, and 36.57% respectively. In Fig. 7(d), even though, we might not be able to notice a big difference among the four models at the beginning. On careful inspection, we can observe that the proposed F-DRL STLC reaches the maximum value first which means it is able to flow the traffic more smoother.

Fig. 7 demonstrates that all the approaches including the state-of-the-art (MMTLC, MPTLC, FTATLC, and F-DRL STLC) aim to clear the traffic in the same set of episodes, due to which overlapping is noticed in the graphs. However, it is observable from the pink line that the F-DRL STLC requires less number of episodes to clear vehicular traffic as compared to other methods. F-DRL STLC performs better than other state-of-the-art methods as it collects traffic information from both the local and neighboring intersections. This holistic view of traffic flow helps to reduce congestion and improve the overall efficiency of traffic flow. Therefore, it can be inferred that the proposed Fog-integrated DRL-based STLC can be a promising solution to reduce congestion at intersections.

VI. CONCLUSION

This work proposes a Fog integrated DRL-based Smart Traffic Light Controller (STLC) cooperative framework to optimize congestion at the intersection by considering the traffic conditions of both local and neighboring intersections. The framework generates a new Cycle Phase Duration (CPD), based on DRL agent that is deployed on Fog node. The CPD is generated, for the STLC, by taking into account average waiting time and queue length at the intersection through appropriate coordination with neighboring intersections. Subsequently, the generated CPD is used as input for the max-pressure based algorithm which is deployed at the intersection. Further, the effectiveness of is evaluated via various vehicle arrival rates, and results are compared with other state-of-the-art methods such as MMTLC [19], MPTLC [3], and FTATLC [20] using an open-source SUMO simulator [18]. The compared results are the evidence for the effectiveness of the proposed method. In future work, we will examine its performance in more realistic traffic scenarios with heavy road traffic and eventually develop a speed prediction model to improve the traffic flow at the intersections.

REFERENCES

- [1] M. Ma, J. A. Stankovic, and L. Feng, "Cityresolver: a decision support system for conflict resolution in smart cities," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPs)*. IEEE, 2018, pp. 55–64.
- [2] M. Balta and I. Özcelik, "Traffic signaling optimization for intelligent and green transportation in smart cities," in *2018 3rd International Conference on Computer Science and Engineering (UBMK)*. IEEE, 2018, pp. 31–35.
- [3] L. Li, R. Li, Y. Peng, C. Huang, and J. Yuan, "Cooperative max-pressure enhanced traffic signal control," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 4173–4177.
- [4] D. Zhang, "Mobile cyber-physical systems for smart cities," in *Companion Proceedings of the Web Conference 2020*, 2020, pp. 546–548.
- [5] L. F. P. de Oliveira, L. T. Manera, and P. D. G. da Luz, "Development of a smart traffic light control system with real-time monitoring," *IEEE Internet of Things Journal*, 2020.
- [6] N. E. Mohamed and I. I. Radwan, "Traffic light control design approaches: a systematic literature review," *International Journal of Electrical & Computer Engineering (2088-8708)*, vol. 12, no. 5, 2022.
- [7] A. Sachan and N. Kumar, "Intelligent traffic control system for emergency vehicles," in *IoT and Analytics for Sensor Networks*. Springer, 2022, pp. 151–160.
- [8] V. Garg, A. Sachan, N. Kumar, and S. Mittal, "Congestion control utilizing software defined control architecture at the traffic light intersection," in *2021 IEEE 18th International Conference on Mobile Ad Hoc and Smart Systems (MASS)*. IEEE, 2021, pp. 597–602.
- [9] P. Mercader, W. Uwayid, and J. Haddad, "Max-pressure traffic controller based on travel times: An experimental analysis," *Transportation Research Part C: Emerging Technologies*, vol. 110, pp. 275–290, 2020.
- [10] A. Sachan and N. Kumar, "Sdn control-enabled and time-quantum-driven max-pressure approach for intersection management in smart city," *IEEE Systems Journal*, 2022.
- [11] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0968090X13001782>
- [12] H. Wei, G. Zheng, V. Gayah, and Z. Li, "Recent advances in reinforcement learning for traffic signal control: A survey of models and evaluation," *ACM SIGKDD Explorations Newsletter*, vol. 22, no. 2, pp. 12–18, 2021.
- [13] M. Abdoos and A. L. Bazzan, "Hierarchical traffic signal optimization using reinforcement learning and traffic prediction with long-short term memory," *Expert systems with applications*, vol. 171, p. 114580, 2021.
- [14] U. Baumgart and M. Burger, "A reinforcement learning approach for traffic control," in *VEHITS*, 2021, pp. 133–141.
- [15] N. Kumar, S. S. Rahman, and N. Dhakad, "Fuzzy inference enabled deep reinforcement learning-based traffic light control for intelligent transportation system," *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [16] N. Kumar, S. Mittal, V. Garg, and N. Kumar, "Deep reinforcement learning-based traffic light scheduling framework for sdn-enabled smart transportation system," *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [17] A. Boukerche, D. Zhong, and P. Sun, "A novel reinforcement learning-based cooperative traffic signal system through max-pressure control," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 2, pp. 1187–1198, 2021.
- [18] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, "Recent development and applications of SUMO - Simulation of Urban MObility," *International Journal On Advances in Systems and Measurements*, vol. 5, no. 3&4, pp. 128–138, December 2012. [Online]. Available: <http://elib.dlr.de/80483/>
- [19] D. Jutury, N. Kumar, A. Sachan, Y. Daultani, and N. Dhakad, "Adaptive neuro-fuzzy enabled multi-mode traffic light control system for urban transport network," *Applied Intelligence*, pp. 1–22, 2022.
- [20] J. Wahlstedt, "Evaluation of the two self-optimising traffic signal systems utopia/spot and imflow, and comparison with existing signal control in stockholm, sweden," in *16th International IEEE Conference on Intelligent Transportation Systems (ITSC 2013)*, 2013, pp. 1541–1546.
- [21] B. Kamasetty, M. Renduchintala, L. L. Shetty, S. Chandarshekar, and R. Shettar, "Design and development of portable smart traffic signaling system with cloud-artificial intelligence enablement," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 26, no. 1, pp. 116–126, 2022.
- [22] B. Zheng, C.-W. Lin, S. Shiraishi, and Q. Zhu, "Design and analysis of delay-tolerant intelligent intersection management," *ACM Transactions on Cyber-Physical Systems*, vol. 4, no. 1, pp. 1–27, 2019.
- [23] Vignesh.Viswanathan and V. Santhanam, "Intelligent traffic signal control using wireless sensor networks," 2013.
- [24] G. Como and G. Nilsson, "On the well-posedness of deterministic queueing networks with feedback control," *Transportation Research Part B: Methodological*, vol. 150, pp. 323–335, 2021.
- [25] D. F. Borges, J. P. R. Leite, E. M. Moreira, and O. A. Carpinteiro, "Traffic light control using hierarchical reinforcement learning and options framework," *IEEE Access*, vol. 9, pp. 99 155–99 165, 2021.
- [26] J. Wang, H. Zhao, H. Liu, L. Geng, and Z. Sun, "A distributed vehicle-assisted computation offloading scheme based on drl in vehicular networks," in *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2022, pp. 200–209.
- [27] A. Staffolani, V.-A. Darvari, P. Bellavista, and M. Musolesi, "RLq: Workload allocation with reinforcement learning in distributed queues," *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [28] G. (general-purpose input/output) pins. [Online]. Available: <https://www.raspberrypi.org/documentation/usage/gpio>